FACE MASK DETECTION

CNN + OpenCV Complete Project Documentation

MaskCNN_v2 | Keras + TensorFlow + OpenCV

Model: MaskCNN_v2 | Version: 2.0.0 | Date: May 2026
Training: Google Colab (T4 GPU) | Testing: PyCharm + OpenCV

Test Accuracy: 96.43% | ROC-AUC: 0.9939 | Params: 315,793

Table of Contents

Table of Contents 2
Executive Summary 5
1. Theoretical Foundations 6
1.1 Introduction to Neural Networks for Image Recognition 6
1.2 Fully Connected Networks and Their Limitations 6
1.3 Convolutional Neural Networks (CNN) 6
1.3.1 Convolution Layers 6
1.3.2 Receptive Fields and Hierarchical Feature Learning 7
1.3.3 Pooling Layers 7
1.3.4 Batch Normalization 7
1.3.5 Residual Skip Connections 7
1.3.6 Global Average Pooling 8
1.3.7 Dropout and Spatial Dropout 8
1.4 OpenCV — Computer Vision Library 8
1.4.1 Image Processing Pipeline 8
2. Project Overview 10
2.1 Problem Definition 10
2.2 System Architecture Overview 10
2.3 File Structure 10
2.4 Technology Stack 11
3. Dataset Analysis 12
3.1 Dataset Source and Structure 12
3.2 Image Characteristics 12
3.3 Class Balance and Weighting 12
3.4 Data Augmentation Strategy 12
3.4.1 Pipeline Architecture 13
4. CNN Architecture — MaskCNN_v2 14
4.1 Design Philosophy 14
4.2 Layer-by-Layer Architecture 14
4.3 Residual Skip Connections — Implementation 14
4.4 Mixed Precision Training 15
4.5 Regularization Strategy 15
5. Hyperparameter Optimization 16
5.1 Why Hyperparameter Optimization? 16
5.2 KerasTuner Hyperband Algorithm 16

5.3 Search Space Definition 16
5.4 Winning Hyperparameters 17
6. Model Training 18
6.1 Learning Rate Schedule — WarmupCosineSchedule 18
Linear Warmup Phase (Epochs 1-5) 18
Cosine Decay Phase (Epochs 6-30) 18
6.2 Training Configuration 18
6.3 Callbacks 19
6.4 Training Results Summary 19
7. Model Evaluation and Results 20
7.1 Key Performance Metrics 20
7.2 Validation Set Metrics 20
7.3 Test Set Metrics — Unbiased Final Evaluation 20
7.4 Interpretation of Results 21
7.4.1 Validation vs Test Gap 21
7.4.2 Precision-Recall Tradeoff 21
7.4.3 ROC-AUC Interpretation 21
7.4.4 Threshold Analysis 21
7.5 Grad-CAM Explainability 21
7.6 MaxPool Feature Map Analysis 22
8. OpenCV Integration and Real-Time Detection 23
8.1 Face Detection — OpenCV DNN SSD 23
8.1.1 SSD Detection Pipeline 23
8.2 Mask Classification Inference Pipeline 23
8.3 Model Loading with Custom Object 24
9. PyCharm Testing — 12 Experiments 25
9.1 Setup and Architecture 25
Core Setup (Cells 1-4) 25
9.2 Experiments Overview 25
9.3 Experiment Deep Dives 26
Experiment 4 — Threshold Slider 26
Experiment 5 — Grayscale Robustness 26
Experiment 12 — Alarm Mode 26
9.4 The Annotate Frame Function 26
Sigmoid Semantics 26
10. Engineering Decisions and Bug Fixes 28

10.1 Critical Bug Fixes Applied in v2 28
10.2 Performance Optimization Decisions 28
10.2.1 Dataset Copy to Local SSD 28
10.2.2 Batch Inference in Real-Time Pipeline 29
10.2.3 OpenCV DNN SSD over Haar Cascade 29
11. Model Configuration and Persistence 30
11.1 config.json Structure 30
11.2 Model Files 30
12. ML Engineering Checklist 31
12.1 Production Readiness Assessment 31
12.2 Limitations and Future Improvements 31
13. References and Resources 32
13.1 Dataset 32
13.2 Frameworks and Libraries 32
13.3 Key Papers and Concepts 32

Executive Summary

This document provides comprehensive technical documentation for a real-time Face Mask Detection system built from scratch using Convolutional Neural Networks (CNN) and OpenCV. The project was developed as a complete ML engineering exercise, covering the full pipeline from dataset analysis and model training on Google Colab to live webcam testing on PyCharm.

Project Highlights

Architecture: Custom CNN (MaskCNN_v2) with residual skip connections, BatchNorm, and SpatialDropout, built from scratch in Keras.

HPO: KerasTuner Hyperband search over 38 trials (84 min) identifying optimal hyperparameters.

Training: Adam + Warmup Cosine LR Schedule, mixed float16 precision on Google Colab T4 GPU.

Performance: 96.43% test accuracy, ROC-AUC 0.9939, only 26 misclassifications on 728 test images.

Deployment: OpenCV DNN SSD face detector + Keras mask classifier in a real-time pipeline running on CPU.

Explainability: Grad-CAM heatmaps and MaxPool feature map visualizations confirm the model focuses on the nose/mouth region.

1. Theoretical Foundations

1.1 Introduction to Neural Networks for Image Recognition

Human brains can identify objects in images in under 100 milliseconds — a feat that seems effortless but requires extraordinarily complex processing. Artificial neural networks, inspired by the structure of the human brain, have demonstrated that machines can learn to perform similar tasks with remarkable accuracy. Image recognition has become one of the most popular applications of deep learning, and in recent years we have witnessed unprecedented progress in this field.

1.2 Fully Connected Networks and Their Limitations

A traditional fully connected (dense) neural network assigns a weight to every connection between every neuron in adjacent layers. While powerful for many tasks, this architecture poses serious challenges for image recognition:

- Parameter Explosion: A simple 50x50 pixel RGB image fed into a network with just one hidden layer of 2,000 neurons requires approximately 15 million trainable weights. Real-world images (128x128 or larger) make this computationally infeasible.
- No Spatial Awareness: Fully connected layers treat every pixel independently, losing all spatial relationships. They cannot exploit the fact that nearby pixels in an image are often related.
- Poor Generalization: The massive parameter count makes these networks prone to overfitting, especially on smaller datasets.

Parameter Count Example

50x50 pixel RGB image input = 7,500 neurons

One hidden layer = 2,000 neurons

Total connections: (7,500 x 2,000) + (2,000 x 2) = 15,004,000 weights

This is for a tiny image. At 128x128, it scales to ~100 million parameters — all from one image.

1.3 Convolutional Neural Networks (CNN)

CNNs solve the scaling problem by exploiting the spatial structure of images. Instead of connecting every neuron to every other neuron, CNNs use shared convolutional filters (kernels) that slide across the image, dramatically reducing the number of parameters while preserving spatial relationships.

1.3.1 Convolution Layers

The core building block of a CNN is the convolution layer. A kernel — a small square window of learnable weights — is systematically scanned across the entire input image. At each position, a dot product is computed between the kernel weights and the pixel values in the window. The

result is a feature map describing the presence of a particular pattern (edge, curve, texture) at every location in the image.

- The kernel weights are learned during training — the network decides which patterns are important.
- Each convolutional layer can contain dozens or hundreds of different kernels, each producing its own feature map.
- Early layers detect simple patterns (edges, corners); deeper layers detect complex structures (eyes, masks, faces).

1.3.2 Receptive Fields and Hierarchical Feature Learning

A key insight of CNNs is the concept of the receptive field — the region of the original input image that influences a given neuron. In early layers, a 3x3 kernel sees only 9 pixels. After a pooling operation, the same kernel in the next layer effectively sees 4x the input area. As depth increases, each neuron can "see" progressively larger portions of the image, enabling detection of increasingly complex patterns.

1.3.3 Pooling Layers

Pooling layers perform spatial downsampling after convolution. Max pooling, the most common variant, replaces a local neighborhood with its maximum value. This achieves two goals:

- Dimensionality Reduction: Reduces the spatial size of feature maps, lowering computational cost.
- Translational Invariance: Makes the model robust to small shifts in the position of a detected feature — the mask need not be in exactly the same position in every training image.

1.3.4 Batch Normalization

Batch Normalization (BN) normalizes the activations of each layer across the mini-batch, stabilizing training and allowing for higher learning rates. In our architecture, BN is applied after every convolutional layer before the ReLU activation. This was a critical design decision that improved convergence speed and final accuracy.

1.3.5 Residual Skip Connections

Introduced by ResNet (He et al., 2016), residual connections add the input of a block directly to its output. This means the block only needs to learn the residual (the difference) rather than the full mapping. Benefits include:

- Mitigates the vanishing gradient problem in deep networks.
- Allows gradients to flow directly to earlier layers, enabling effective training of deeper architectures.
- Our MaskCNN_v2 applies residual skips in Blocks 2 and 3 using 1x1 convolutions to match channel dimensions.

1.3.6 Global Average Pooling

Instead of flattening feature maps into a long vector (which loses spatial information and introduces many parameters), Global Average Pooling (GAP) computes the mean of each entire feature map. This drastically reduces parameters and acts as a structural regularizer, making the network more robust.

1.3.7 Dropout and Spatial Dropout

Dropout randomly sets a fraction of neurons to zero during training, preventing co-adaptation and reducing overfitting. SpatialDropout2D, used in our convolutional blocks, drops entire feature maps rather than individual neurons — more appropriate for convolutional layers where adjacent activations are highly correlated.

1.4 OpenCV — Computer Vision Library

OpenCV (Open Source Computer Vision Library) is the industry-standard library for computer vision tasks. It provides a comprehensive suite of tools organized across multiple modules:

Module	Purpose	Used In This Project
core	Fundamental data structures (Mat, arrays)	Image storage and manipulation
imgproc	Image processing operations	Resizing, color conversion, blur
imgcodecs	Image file I/O	Loading images from disk
highgui	Display windows and trackbars	All real-time experiment windows
video	Video I/O and motion analysis	Webcam capture, video writing
dnn	Deep Neural Network inference	SSD face detector integration
features2d	Feature detection and matching	Edge analysis (Exp 8)

1.4.1 Image Processing Pipeline

OpenCV processes images through three levels:

- Low-level: Pixel-level operations — noise removal, sharpening, illumination normalization, perspective correction.
- Mid-level: Structural extraction — converting the processed image into a numerical representation (e.g., bounding box coordinates for a detected face).
- High-level: Semantic understanding — face recognition, emotion detection, mask classification.

OpenCV uses BGR color order (not RGB). When interfacing with Keras/TensorFlow (which expects RGB), color conversion is mandatory:

```
rgb = cv2.cvtColor(bgr_image, cv2.COLOR_BGR2RGB)
```

2. Project Overview

2.1 Problem Definition

The COVID-19 pandemic created an urgent need for automated systems to enforce mask-wearing mandates in public spaces. Manual enforcement is expensive, inconsistent, and introduces health risks for enforcement personnel. An automated, camera-based system can provide consistent, real-time monitoring without human intervention.

This project implements a binary image classifier that determines, for each detected face in a camera frame, whether the person is wearing a mask ("with_mask") or not ("without_mask"). The system must be fast enough for real-time use on standard hardware.

2.2 System Architecture Overview

The complete system consists of two decoupled stages:

Stage	Component	Tool	Environment
Stage 1: Face Detection	OpenCV DNN SSD Face Detector	OpenCV + Caffe model	Real-time inference
Stage 2: Mask Classification	MaskCNN_v2	Keras/TensorFlow	128x128 RGB input
Training Environment	Hyperband HPO + Final Training	Google Colab T4 GPU	Cloud
Testing Environment	Live webcam + 12 experiments	PyCharm + CPU	Local
Model Persistence	config.json + .keras + .h5	JSON + HDF5	File system

2.3 File Structure

The project consists of the following files:

- mask_detection_CNN_grid_search.ipynb — Initial training notebook with full KerasTuner Hyperband search across 38 trials, complete evaluation pipeline, Grad-CAM, and feature map visualization.
- mask_detection_CNN_winning_params.ipynb — Final training notebook with winning hyperparameters hardcoded from the completed search. Clean, reproducible re-run without HPO overhead.
- mask_detector_pycharm.ipynb — OpenCV real-time testing notebook for PyCharm. Implements 12 distinct experiments covering basic detection, robustness tests, and alarm systems.
- mask_cnn_final.keras — Full serialized Keras model including architecture, weights, and optimizer state.
- best_mask_cnn_weights.h5 — Best validation checkpoint weights (saved at epoch 25).

- deploy.prototxt — OpenCV DNN SSD face detector architecture definition (Caffe format).
- config.json — Complete model configuration: hyperparameters, metrics, class map, threshold, inference settings.

2.4 Technology Stack

Technology	Version/Detail	Role
TensorFlow/Keras	2.x (mixed float16)	Model building, training, inference
KerasTuner	Latest	Hyperparameter optimization (Hyperband)
OpenCV	4.x	Face detection, webcam I/O, annotation
NumPy	Latest	Array operations, metric computation
Scikit-learn	Latest	Evaluation metrics, confusion matrices, class weights
Matplotlib/Seaborn	Latest	Training curves, ROC, visualization
Google Colab T4 GPU	CUDA-enabled	Training acceleration
Python	3.x	Primary language

3. Dataset Analysis

3.1 Dataset Source and Structure

The dataset used is the Kaggle Face Mask Detection dataset by Omkar Gurav, comprising 7,158 images pre-split into train, validation, and test sets. The images contain cropped face regions labeled as either "with_mask" or "without_mask".

Split	With Mask	Without Mask	Total	Proportion
Train	~2,648	~2,648	5,296	74.0%
Validation	~567	~567	1,134	15.8%
Test	167	561	728	10.2%
TOTAL	~3,382	~3,776	7,158	100%

3.2 Image Characteristics

A scan of the first 200 images per class in the training set revealed:

- Variable dimensions: heights and widths ranging from small crops to larger face images.
- All images standardized to 128x128 pixels (RGB) before model input.
- Both JPEG and PNG formats present — requiring decode_image() rather than decode_jpeg() to avoid format-specific crashes.
- 3-channel RGB images throughout (no grayscale or RGBA).

3.3 Class Balance and Weighting

The training set is near-perfectly balanced between the two classes. However, to handle any residual imbalance and improve robustness, class weights were computed using scikit-learn's compute_class_weight with balanced mode:

Computed Class Weights

Class 0 (with_mask): weight = 1.0134

Class 1 (without_mask): weight = 0.9870

Weights close to 1.0 confirm near-perfect balance. Slight adjustment prevents trivial solutions.

3.4 Data Augmentation Strategy

Augmentation was applied exclusively to the training split using a GPU-accelerated tf.data + tf.image pipeline. The validation and test sets remained untouched to provide unbiased evaluation. The strategy of never showing the model the exact same image twice exposes it to more aspects of the data, improving generalization.

Transform	Parameters	Purpose
Random Horizontal Flip	50% probability	Handles left/right facing people
Random Brightness	max_delta = 0.30	Simulates different lighting conditions
Random Contrast	lower=0.8, upper=1.2	Handles over/under-exposed cameras
Random Zoom/Crop	scale 0.80 to 1.0	Simulates different distances from camera
Random Rotation	factor=0.056 (±20°)	Handles tilted faces and camera angles
Pixel Rescale	divide by 255.0	Normalizes to [0, 1] float32 range

3.4.1 Pipeline Architecture

The tf.data pipeline follows an optimized order designed to maximize GPU utilization while minimizing CPU bottlenecks:

```
shuffle(buffer) → list_files → load_image (decode + resize + normalize)
→ cache (in memory) → augment (GPU) → batch(64) → prefetch(AUTOTUNE)
```

The critical optimization that reduced training step time from 15 seconds to 0.08 seconds was copying the dataset from Google Drive's network mount to Colab's local SSD before training. Network I/O was the bottleneck, not compute.

4. CNN Architecture — MaskCNN_v2

4.1 Design Philosophy

The MaskCNN_v2 was designed from scratch with three guiding principles:

1. Efficiency: The model must be small enough for real-time CPU inference (target <50ms per frame).
2. Robustness: Residual connections, batch normalization, and regularization prevent overfitting on the 5,296-image training set.
3. Trainability: Mixed precision (float16) and a WarmupCosine learning rate schedule ensure stable, fast convergence.

4.2 Layer-by-Layer Architecture

The final model uses 3 active convolutional blocks (Block 4 disabled by the HPO search) plus a classification head, totaling 315,793 trainable parameters.

Block	Layers	Filters	Output Shape	Notes
Input	keras.Input	—	128 x 128 x 3	Float32 normalized [0,1]
Block 1	Conv2D → BN → ReLU (x2) → MaxPool2D	16 filters	64 x 64 x 16	No skip — first block
Block 2	Conv2D → BN → ReLU (x2) → Add → MaxPool2D	64 filters	32 x 32 x 64	Residual skip via 1x1 Conv
Block 3	Conv2D → BN → ReLU (x2) → Add → MaxPool2D	128 filters	16 x 16 x 128	Residual skip via 1x1 Conv
Block 4	(Disabled)	256 filters	—	use_block4=False from HPO
SpatialDropout	SpatialDropout2D(0.25)	—	16 x 16 x 128	Drops entire feature maps
Head	GlobalAvgPool2D → Dense(256) → BN → Dropout(0.6)	—	256	Dense classifier
Output	Dense(1, sigmoid) cast float32	—	1	P(without_mask)

4.3 Residual Skip Connections — Implementation

Blocks 2 and 3 implement residual connections. Since the number of filters changes between the skip path and the main path, a 1x1 convolution projects the skip tensor to the correct channel dimension before addition:

```
# Block 2 example
skip2 = Conv2D(filters_2, kernel_size=1, padding='same')(x)  # match channels
```

```
x = Conv2D(filters_2, 3, padding='same')(x)
x = BatchNormalization()(x)
x = Activation('relu')(x)
x = Conv2D(filters_2, 3, padding='same')(x)
x = BatchNormalization()(x)
x = Add()([x, skip2])   # residual addition
x = Activation('relu')(x)
x = MaxPooling2D(2)(x)
```

4.4 Mixed Precision Training

Mixed precision (float16 compute, float32 parameter storage) was enabled globally before model construction:

```
tf.keras.mixed_precision.set_global_policy('mixed_float16')
```

Benefits: approximately 2x memory reduction, significantly faster matrix multiplications on T4 GPU's Tensor Cores. The output sigmoid layer explicitly casts to float32 to avoid numerical instability in the loss computation.

4.5 Regularization Strategy

Multiple complementary regularization techniques prevent overfitting:

Technique	Configuration	Applied To
L2 Weight Decay	lambda = 1e-4	All Conv2D kernel weights
SpatialDropout2D	rate = 0.25	After Block 3 feature maps
Dense Dropout	rate = 0.60	Between Dense head layers
Label Smoothing	epsilon = 0.10	Binary cross-entropy loss
Data Augmentation	5 transforms	Training images only
Class Weighting	Balanced	Training loss scaling

Label smoothing (ε=0.1) prevents the model from becoming overconfident. Instead of training toward hard targets of 0 and 1, it trains toward 0.05 and 0.95, encouraging calibrated probability outputs.

5. Hyperparameter Optimization

5.1 Why Hyperparameter Optimization?

A CNN's performance depends critically on choices that cannot be derived analytically: the number of filters per layer, dropout rates, learning rate, regularization strength, and whether to use optional components like Block 4. Manual tuning of these values is both time-consuming and biased. Automated HPO searches the configuration space systematically, finding combinations that a human might never try.

5.2 KerasTuner Hyperband Algorithm

Hyperband is an advanced multi-armed bandit algorithm that efficiently allocates computational resources by eliminating poor-performing configurations early. The algorithm works through successive halving:

4. Start all trials with a small budget (few epochs).
5. After each round, keep only the top 1/factor (factor=3) configurations.
6. Promote survivors to the next round with a larger epoch budget.
7. Repeat until one configuration wins.

Compared to random or grid search, Hyperband finds good configurations much faster by spending less time on configurations that quickly reveal poor performance. In our project, Hyperband ran with max_epochs=15, factor=3, and hyperband_iterations=2 — completing 38+ trials in 84 minutes.

5.3 Search Space Definition

Hyperparameter	Choices Explored	Type
filters_1	16, 32	Conv2D filter count (Block 1)
filters_2	32, 64	Conv2D filter count (Block 2)
filters_3	64, 128	Conv2D filter count (Block 3)
filters_4	128, 256	Conv2D filter count (Block 4)
dense_units	128, 256, 512	Dense head neuron count
dropout_1	0.25, 0.40, 0.50	SpatialDropout2D rate
dropout_2	0.40, 0.50, 0.60	Dense Dropout rate
learning_rate	1e-3, 5e-4, 1e-4	Initial Adam LR
l2_reg	0.0, 1e-4, 1e-3	L2 weight decay lambda
use_block4	True, False	Whether to include Block 4
label_smoothing	0.0, 0.1	Loss label smoothing epsilon

5.4 Winning Hyperparameters

After 38 trials and 84 minutes, the Hyperband search identified the following optimal configuration (best val_accuracy = 0.9700, Trial 38):

Hyperparameter	Winning Value	Interpretation
filters_1	16	Lighter first block — fewer edge detectors needed
filters_2	64	Standard capacity for mid-level features
filters_3	128	High capacity for complex facial features
filters_4	256	Unused (use_block4=False)
dense_units	256	Balanced classification head
dropout_1 (spatial)	0.25	Light regularization of feature maps
dropout_2 (dense)	0.60	Strong regularization of dense layer
learning_rate	1e-3	Aggressive initial LR — cosine warmup manages it
l2_reg	1e-4	Light weight decay throughout
use_block4	False	Block 4 disabled — better generalization, less RAM
label_smoothing	0.10	Prevents overconfidence

Key Insight from HPO

The most surprising finding was use_block4=False. Adding a 4th convolutional block with 256 filters appeared to hurt generalization on this dataset size (5,296 training images). The model with 3 blocks (315K parameters) outperformed the 4-block variant despite having fewer parameters — a textbook example of capacity vs. overfitting tradeoffs.

6. Model Training

6.1 Learning Rate Schedule — WarmupCosineSchedule

Instead of a fixed learning rate or simple decay, we implemented a custom Warmup + Cosine Decay schedule. This is a production-grade approach used in state-of-the-art models:

Linear Warmup Phase (Epochs 1-5)

The learning rate linearly increases from near-zero to the target rate (1e-3) over the first 5 epochs. This prevents the optimizer from making large, destabilizing updates early in training when the randomly initialized weights are far from their optimal values.

```
warmup_lr = initial_lr * (step / warmup_steps)
```

Cosine Decay Phase (Epochs 6-30)

After warmup, the learning rate follows a cosine curve from the peak value down to a near-zero minimum (alpha=1e-7). Cosine decay is smooth and avoids the abrupt drops of step decay, allowing fine-grained weight adjustments in later epochs.

```
cosine_lr = initial_lr * (alpha + (1-alpha) * 0.5 * (1 + cos(pi * progress)))
```

The schedule was registered as a Keras-serializable class with @register_keras_serializable, enabling the model to be saved and loaded without losing the optimizer state.

6.2 Training Configuration

Parameter	Value	Rationale
Optimizer	Adam	Adaptive learning rates for each parameter
LR Schedule	WarmupCosineSchedule	Stable convergence, avoids early divergence
Loss Function	BinaryCrossentropy(label_smoothing=0.1)	Calibrated probabilities, prevents overconfidence
Epochs	30 (best at epoch 25)	Early stopping patience=8 applied
Batch Size	64	Good GPU utilization on T4
Mixed Precision	float16	2x memory efficiency, faster compute
Class Weights	{0: 1.0134, 1: 0.9870}	Balanced class contribution
Metrics	accuracy, precision, recall, AUC	Comprehensive training monitoring

6.3 Callbacks

Callback	Configuration	Purpose
ModelCheckpoint	monitor=val_loss, save_best_only=True	Saves weights at best validation epoch
EarlyStopping	monitor=val_loss, patience=8	Stops training if no improvement for 8 epochs
CSVLogger	LOG_PATH	Records all metrics per epoch to CSV

Both ModelCheckpoint and EarlyStopping monitor val_loss (not val_accuracy). Using the same monitor eliminates a subtle bug from v1 where the checkpoint was saving based on one metric while early stopping was based on another — a common source of unexpected behavior in training pipelines.

6.4 Training Results Summary

Training Outcome

Total Epochs Run: 30 | Best Epoch: 25 | Total Params: 315,793
Training step time: ~0.08 seconds (after dataset copy to local SSD)
Total training time: approximately 2.4 minutes (30 epochs × 82 steps × 0.08s)

Best Epoch 25 Metrics:
Train Loss: ~0.12 | Val Loss: ~0.09
Train Accuracy: ~97.5% | Val Accuracy: 97.7%
Train AUC: ~0.998 | Val AUC: 0.9963

7. Model Evaluation and Results

7.1 Key Performance Metrics

96.43%	0.9939	0.9240	315K
Test Accuracy	**ROC-AUC**	**F1-Score**	**Parameters**
728 images	Test set	Without Mask class	Total trainable

7.2 Validation Set Metrics

The validation set metrics carry a slight optimistic bias due to checkpoint selection — the saved weights correspond to the epoch that performed best on the validation set.

Metric	Value	Note
Accuracy	97.71%	Slight optimistic bias from checkpoint
Precision	98.24%	Of predicted 'without_mask', 98.24% correct
Recall	97.21%	Of actual 'without_mask', 97.21% detected
F1-Score	97.72%	Harmonic mean of precision and recall
ROC-AUC	0.9963	Near-perfect discrimination
Decision Threshold	0.5000	Applied at evaluation

7.3 Test Set Metrics — Unbiased Final Evaluation

The test set was completely held out during all training and monitoring phases. These are the true generalization metrics:

Metric	Value	Breakdown
Accuracy	96.43%	702/728 correct predictions
Precision	90.29%	Of predicted without_mask, 90.29% correct (17 FP)
Recall	94.61%	Of actual without_mask, 94.61% detected (9 FN)
F1-Score	92.40%	Balanced precision/recall for without_mask class
ROC-AUC	0.9939	Excellent discrimination ability
True Positives (TP)	158	without_mask correctly detected
True Negatives (TN)	544	with_mask correctly classified

False Positives (FP)	17	with_mask incorrectly flagged as without_mask
False Negatives (FN)	9	without_mask missed — most critical error type

7.4 Interpretation of Results

7.4.1 Validation vs Test Gap

The gap between validation accuracy (97.71%) and test accuracy (96.43%) is 1.28 percentage points. This small gap is expected and healthy — it reflects the slight optimistic bias from checkpoint selection on the validation set. A larger gap would indicate overfitting; a gap this small confirms good generalization.

7.4.2 Precision-Recall Tradeoff

The test precision (90.29%) is notably lower than recall (94.61%). This means the model is more likely to incorrectly flag a masked person as unmasked (17 false positives) than to miss an unmasked person (9 false negatives). In a real-world deployment context:

- False Negatives (missed threats): 9 cases. An unmasked person passes through undetected — the more serious error type for health enforcement.
- False Positives (unnecessary alarms): 17 cases. A masked person is falsely flagged — causes minor inconvenience.

For public health applications, minimizing false negatives is more critical. The model's higher recall than precision reflects a sensible implicit bias toward safety.

7.4.3 ROC-AUC Interpretation

An AUC of 0.9939 means that for a randomly chosen (without_mask, with_mask) pair, the model correctly assigns a higher probability to the without_mask image 99.39% of the time. This near-perfect discrimination indicates the model has learned a very reliable decision boundary.

7.4.4 Threshold Analysis

A threshold sensitivity analysis was performed on the validation set, sweeping the decision threshold from 0.10 to 0.95 and measuring precision, recall, F1, and accuracy at each value. The analysis confirmed that the default threshold of 0.5 is optimal for this dataset — the F1 curve peaks at exactly 0.5, validating the chosen operating point.

7.5 Grad-CAM Explainability

Gradient-weighted Class Activation Mapping (Grad-CAM) was implemented to verify that the model is making decisions based on the correct visual features. Grad-CAM computes the gradient of the class score with respect to the final convolutional layer's feature maps, then weights those maps by their importance to produce a heatmap.

Results confirmed: the heatmaps concentrate strongly on the nose and mouth region for both classes. For 'with_mask' images, the mask fabric region is highlighted. For 'without_mask' images, the exposed nose/mouth area drives the classification. This confirms the model has learned semantically meaningful features rather than spurious correlations (such as background or clothing).

7.6 MaxPool Feature Map Analysis

Feature maps extracted after each MaxPooling layer provide insight into the hierarchical feature extraction:

- Block 1 MaxPool (64x64x16): Feature maps show edge patterns and basic texture variations. All 16 channels are actively responding to different aspects of the input image.
- Block 2 MaxPool (32x32x64): More abstract representations emerge. Some channels specialize in detecting rounded shapes (mask outlines), others in skin-texture boundaries.
- Block 3 MaxPool (16x16x128): Highly abstract, semantically meaningful features. A notable increase in sparsity is visible — many channels produce near-zero activations except in specific high-level regions, consistent with specialized mask/face detectors.

The increasing sparsity across blocks is a healthy sign of progressive specialization, consistent with the theoretical description of CNN layer hierarchy.

8. OpenCV Integration and Real-Time Detection

8.1 Face Detection — OpenCV DNN SSD

Face detection uses the OpenCV DNN module with a pre-trained Single Shot Multibox Detector (SSD) model using a ResNet-10 backbone, trained by the OpenCV team in 2017. This replaces the older Haar Cascade approach used in v1.

Attribute	OpenCV DNN SSD	Haar Cascade (v1, replaced)
Year	2017	2001
Approach	Deep learning (SSD)	Classical (hand-crafted features)
Angle tolerance	Up to ~70° tilt	Front-facing only
Partial occlusion	Handles well	Fails easily
Speed	~20-30ms per frame	~5ms per frame
Accuracy	High	Moderate
Files needed	deploy.prototxt + .caffemodel	haarcascade_frontalface.xml

8.1.1 SSD Detection Pipeline

```
# Create blob from frame (resize to 300x300, normalize with mean subtraction)
blob = cv2.dnn.blobFromImage(frame, 1.0, (300,300), (104, 177, 123))
face_net.setInput(blob)
detections = face_net.forward()  # shape: (1, 1, N, 7)
# detections[0,0,i,2] = confidence | detections[0,0,i,3:7] = normalized box coords
# Filter by confidence > 0.5, then scale to frame dimensions
```

8.2 Mask Classification Inference Pipeline

For each detected face bounding box, the following pipeline produces a mask/no-mask prediction:

8. Crop: Extract the face region from the BGR frame using the bounding box.
9. Convert: BGR to RGB (cv2.cvtColor with COLOR_BGR2RGB).
10. Resize: Scale to 128x128 pixels (model input size).
11. Normalize: Divide by 255.0, converting uint8 [0,255] to float32 [0,1].
12. Batch: Add batch dimension (expand_dims at axis 0) → shape (1, 128, 128, 3).
13. Infer: MODEL(batch, training=False) → scalar sigmoid output P(without_mask).
14. Classify: Compare to threshold (0.5). Output label + confidence.

Batch inference was implemented for efficiency — when multiple faces are present, all face crops are concatenated into a single batch tensor and processed in one model call, reducing overhead compared to processing faces one at a time.

8.3 Model Loading with Custom Object

The saved Keras model uses a custom WarmupCosineSchedule learning rate schedule embedded in the optimizer. Loading requires registering this class before calling load_model:

```
@tf.keras.utils.register_keras_serializable(package='MaskCNN')
class WarmupCosineSchedule(LearningRateSchedule):
    ...  # exact copy from training notebook

model = keras.models.load_model(
    'mask_cnn_final.keras',
    custom_objects={'WarmupCosineSchedule': WarmupCosineSchedule}
)
```

9. PyCharm Testing — 12 Experiments

9.1 Setup and Architecture

The PyCharm testing notebook (mask_detector_pycharm.ipynb) implements 12 progressively sophisticated experiments using the trained model and OpenCV. The notebook is structured to be run cell-by-cell, with each experiment self-contained and independently executable after the core setup cells.

Core Setup (Cells 1-4)

All experiments share three foundational components loaded in the setup phase:

- MODEL: The trained MaskCNN_v2 loaded with custom WarmupCosineSchedule.
- FACE_NET: OpenCV DNN SSD face detector (deploy.prototxt + res10_300x300_ssd.caffemodel).
- CFG: Configuration dictionary parsed from config.json (threshold, class map, input size).

9.2 Experiments Overview

Exp	Name	Purpose	Key Feature
1	Basic Live Detection	Standard detection with green/red bounding boxes	Baseline functionality
2	FPS Counter	Real-time frames-per-second display	Performance benchmarking
3	Confidence Bar Graph	Mini horizontal bars below each face	Visual uncertainty display
4	Threshold Slider	Trackbar to adjust decision threshold live	Interactive classification boundary
5	Grayscale Robustness	Strip color from face before inference	Color dependency test
6	Blur Robustness	Gaussian blur on face region, adjustable kernel	Low-resolution simulation
7	Brightness & Contrast	Two trackbars for image adjustment	Lighting variation robustness
8	Side-by-Side Canny	Original vs. Canny edge view split-screen	Low-level feature visualization
9	Session Statistics	Per-session aggregate statistics logger	Production monitoring simulation
10	Video File Testing	Process .mp4 files, save annotated output	Reproducible offline testing

11	Snapshot Chart	Press S to freeze frame and plot bar chart	Presentation tool
12	Alarm Mode	Flashing red border when no mask detected	Access control simulation

9.3 Experiment Deep Dives

Experiment 4 — Threshold Slider

The threshold trackbar experiment is pedagogically the most valuable. It demonstrates in real time how moving the sigmoid decision threshold from 0.0 to 1.0 affects classification. At threshold=0.1, almost all predictions become 'without_mask' (high recall, low precision). At threshold=0.9, almost all become 'with_mask'. The experiment makes the abstract concept of threshold tuning viscerally tangible.

Experiment 5 — Grayscale Robustness

This experiment feeds the model grayscale versions of face crops (R=G=B=gray value) while keeping the DNN face detector on the color frame. The test reveals whether the CNN learned color-dependent features (e.g., using mask color as a cue) or purely texture/shape features. A robust, generalizing model should maintain reasonable accuracy on grayscale input since masks are distinguished primarily by shape and texture (covering the nose/mouth area), not color.

Experiment 12 — Alarm Mode

The most practically relevant experiment simulates a real-world access control system. A flashing red border and 'MASK REQUIRED' text overlay appear whenever ANY detected face in the frame is classified as without_mask. This demonstrates how the binary classifier can be integrated into a larger alarm/notification system — a direct application of the entire pipeline in a production-like scenario.

9.4 The Annotate Frame Function

All experiments share the annotate_frame() helper which applies consistent visual overlays:

- Green bounding box + label (e.g., 'With Mask 97.3%') for masked faces.
- Red bounding box + semi-transparent red fill overlay for unmasked faces.
- Filled label background rectangle for readability against any background color.
- White text on colored background for high contrast.

Sigmoid Semantics

Understanding raw sigmoid output:

raw sigmoid = P(without_mask) [CLASSES[1]]
1 - raw = P(with_mask) [CLASSES[0]]
If raw > threshold (0.5): predict 'without_mask', confidence = raw
If raw <= threshold (0.5): predict 'with_mask', confidence = 1 - raw
The confidence bar (Exp 3) uses these exact formulas for the green and red fills.

10. Engineering Decisions and Bug Fixes

10.1 Critical Bug Fixes Applied in v2

The development process required systematic debugging of 11 distinct issues, each caused by subtle interactions between TensorFlow's execution model, mixed precision, and serialization requirements. These fixes represent important ML engineering knowledge:

Cell	Issue	Root Cause	Fix Applied
13	RandomRotation crash	Creating tf.Variable inside tf.function trace during augment()	Instantiate _rot_layer once at Python module level, outside any mapped function
13	PNG decode crash	decode_jpeg() fails on PNG files; dataset contains mixed formats	Replace with decode_image(expand_animations=False)
13	random_crop shape error	Size argument could be a Python scalar incompatible with tf.Tensor	Use tf.stack([h, h, 3]) to create proper tensor shape
14	matplotlib imshow crash	mixed_float16 produces float16 arrays; imshow only accepts float32/float64	Cast .numpy().astype(np.float32) before every imshow call
19	AttributeError on optimizer LR	optimizer.learning_rate.assign() fails when optimizer holds a schedule object	Pass hp_lr into build_mask_cnn() and compile inside the function
22	Model save fails	WarmupCosineSchedule.get_config() calls .numpy() during serialization (unavailable)	Store all values as plain Python int/float in __init__
22	load_model fails	Custom LR schedule not registered in Keras serialization registry	Add @register_keras_serializable(package='MaskCNN') decorator
31	Feature map dtype error	Feature map tensors are float16 due to mixed precision; matplotlib rejects them	Cast .astype(np.float32) after every .numpy() call on model outputs
33	Grad-CAM returns None	conv_out is a Tensor not a Variable; GradientTape doesn't watch it automatically	Add tape.watch(conv_out) inside the GradientTape context
37	DNN model URL 404	opencv_3rdparty raw URL changed location	Use github.com/opencv/opencv_3rdparty/raw/... with gdown fallback
38	load_model unknown object	WarmupCosineSchedule not passed to custom_objects dict	Pass custom_objects={'WarmupCosineSchedule': WarmupCosineSchedule}

10.2 Performance Optimization Decisions

10.2.1 Dataset Copy to Local SSD

The single highest-impact optimization was copying the dataset from Google Drive's network mount to Colab's local SSD before training. Google Drive network I/O introduced 15 seconds per training step. After the copy, steps ran in 0.08 seconds — a 187x speedup. This is a critical production lesson: always profile your data loading pipeline before optimizing your model.

10.2.2 Batch Inference in Real-Time Pipeline

In the PyCharm testing notebook, the predict_batch() function concatenates all detected face crops into a single NumPy array and calls model() once for the entire batch. This is significantly more efficient than calling predict_face() in a loop, since model overhead (graph setup, memory allocation) occurs once rather than per-face.

10.2.3 OpenCV DNN SSD over Haar Cascade

The DNN face detector was chosen over Haar Cascade despite being slightly slower (~20ms vs ~5ms) because it handles tilted faces (up to 70°) and partially occluded faces. In realistic mask-detection scenarios, people may be wearing masks that partially obscure their face at various angles — the DNN detector handles these cases reliably.

11. Model Configuration and Persistence

11.1 config.json Structure

The config.json file serves as the single source of truth for all inference parameters, enabling the PyCharm testing environment to operate independently from the training environment without hardcoded values:

Section	Key Fields	Purpose
model_name / version	MaskCNN_v2 / 2.0.0	Model identity and versioning
architecture	input_shape, conv_blocks, total_params, winning_hyperparameters	Complete architecture spec
training	optimizer, loss, best_epoch, batch_size, hyperband_search results	Training configuration audit trail
augmentation	pipeline, transforms list	Reproducibility documentation
dataset	train/val/test counts, classes	Data provenance
val_metrics	accuracy, precision, recall, f1, roc_auc	Validation performance
test_metrics	accuracy, precision, recall, f1, roc_auc, tp, tn, fp, fn	Unbiased test performance
inference	classes, class_map, decision_threshold, input_size	Runtime inference parameters
files	model path, weights path, log path	File system references

11.2 Model Files

- mask_cnn_final.keras: Full Keras model in the modern .keras format (ZIP archive containing model JSON, weights, and optimizer state). Required for full inference including the optimizer.
- best_mask_cnn_weights.h5: Checkpoint weights from the best validation epoch (epoch 25). These are the weights actually used for evaluation, loaded via model.load_weights() after building the architecture.
- deploy.prototxt: Caffe-format architecture definition for the OpenCV DNN SSD face detector. 28KB text file defining the ResNet-10 SSD network structure.

12. ML Engineering Checklist

12.1 Production Readiness Assessment

Criterion	Status	Details
Model Accuracy	☑ Met	96.43% test accuracy, exceeds typical 95% target
Inference Latency	☑ Met	~20-30ms per frame on CPU (face detect + classify)
Versioning	☑ Enabled	config.json v2.0.0, .keras + .h5 artifacts
Monitoring Hooks	☑ Implemented	Exp 9 session statistics, FPS counter (Exp 2)
Rollback Ready	☑ Ready	best_mask_cnn_weights.h5 checkpoint preserved
Explainability	☑ Implemented	Grad-CAM + MaxPool feature map visualization
Robustness Tests	☑ Completed	Grayscale, blur, brightness, contrast experiments
Documentation	☑ Complete	This document
Drift Detection	⚠ Manual	Threshold slider enables manual monitoring
Automated Retraining	⚠ Not Implemented	Would require CI/CD pipeline
A/B Testing	⚠ Not Implemented	Single model deployment

12.2 Limitations and Future Improvements

- Dataset Size: 5,296 training images is modest. Performance may degrade on domain shifts (different mask types, lighting conditions, ethnicities not well represented in training data).
- Single-stage Pipeline: Face detection and mask classification are sequential — a missed face detection means no mask classification. An end-to-end model (like YOLO) could improve this.
- Binary Classification: The model cannot distinguish between properly worn masks (over nose and mouth) and improperly worn masks (below nose). A multi-class extension would increase practical utility.
- No Drift Detection: The current implementation has no automated mechanism to detect when the distribution of incoming images shifts away from the training distribution.
- GPU Dependency for Training: Training on CPU would take hours. A production pipeline should automate GPU allocation and model retraining triggers.

13. References and Resources

13.1 Dataset

- Gurav, O. (2021). Face Mask Detection Dataset. Kaggle. https://www.kaggle.com/datasets/omkargurav/face-mask-dataset

13.2 Frameworks and Libraries

- TensorFlow / Keras: https://www.tensorflow.org
- KerasTuner: https://keras.io/keras_tuner/
- OpenCV: https://docs.opencv.org
- OpenCV DNN SSD Face Detector: https://github.com/opencv/opencv/tree/master/samples/dnn/face_detector
- Scikit-learn: https://scikit-learn.org

13.3 Key Papers and Concepts

- He, K. et al. (2016). Deep Residual Learning for Image Recognition. CVPR 2016. — Residual connections.
- Ioffe, S. & Szegedy, C. (2015). Batch Normalization. ICML 2015. — BatchNorm layer.
- Selvaraju, R. et al. (2017). Grad-CAM: Visual Explanations from Deep Networks. ICCV 2017. — Explainability.
- Li, L. et al. (2018). Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization. JMLR 2018. — HPO algorithm.
- Loshchilov, I. & Hutter, F. (2017). SGDR: Stochastic Gradient Descent with Warm Restarts. ICLR 2017. — Cosine schedule.
- Müller, R. et al. (2019). When Does Label Smoothing Help? NeurIPS 2019. — Label smoothing.